

User-Defined Squash Function

The squash operator appears in the Z notation glossary but is not built into the fuzz type-checker. However, we can define it ourselves as a user-defined function using an axiomatic specification.

Squash takes a partial function from $\mathbb{N}_1$ (positive naturals) to some type X and compacts it into a sequence by removing gaps in the domain. For example, $\{1 \mapsto a, 3 \mapsto b, 7 \mapsto c\}$ becomes $\langle a, b, c \rangle$.

Example 1 : Axiomatic Definition of Squash

We define squash axiomatically by specifying its key properties using only fuzz-compatible notation.

$$\begin{array}{|l} \hline [X] \\ \hline \hline \text{squashFunc} : (\mathbb{N}_1 \rightarrow X) \rightarrow \text{seq } X \\ \hline \forall f : \mathbb{N}_1 \rightarrow X \bullet \text{ran}(\text{squashFunc}(f)) = \text{ran } f \wedge \#(\text{squashFunc}(f)) = \#(\text{ran } f) \\ \hline \end{array}$$

This axiomatically defines squashFunc (a user-defined version of squash) with two key properties that hold for all partial functions f from $\mathbb{N}_1$ to X : (1) squashFunc preserves the range: $\text{ran}(\text{squashFunc } f) = \text{ran } f$, and (2) the resulting sequence length equals the range cardinality: $\#(\text{squashFunc } f) = \#(\text{ran } f)$. These properties characterize squashFunc as compacting partial functions by removing domain gaps while preserving all elements and their relative order.

Example 2 : Using Squash on a Simple Function

Demonstrate squash on a sparse partial function.

$$\begin{array}{|l} f1 : \mathbb{N}_1 \rightarrow \mathbb{N} \\ s1 : \text{seq } \mathbb{N} \\ \hline f1 = \{1 \mapsto 10, 3 \mapsto 30, 5 \mapsto 50\} \wedge s1 = \text{squashFunc}(f1) \end{array}$$

The result $s1$ is $\langle 10, 30, 50 \rangle$, containing exactly the range elements in the order of their original indices.

Example 3 : Squash with Letters

$$\begin{array}{|l} [Letter] \\ \hline a : Letter \\ b : Letter \\ c : Letter \\ d : Letter \\ f2 : \mathbb{N}_1 \rightarrow Letter \\ s2 : \text{seq } Letter \\ \hline f2 = \{2 \mapsto a, 4 \mapsto b, 7 \mapsto c, 9 \mapsto d\} \wedge s2 = \text{squashFunc}(f2) \end{array}$$

The result $s2$ is $\langle a, b, c, d \rangle$. The gaps at positions 1, 3, 5, 6, 8 are removed.

Example 4 : Empty Function

$$\begin{array}{|l} f3 : \mathbb{N}_1 \rightarrow \mathbb{N} \\ s3 : \text{seq } \mathbb{N} \\ \hline f3 = \{\} \wedge s3 = \text{squashFunc}(f3) \end{array}$$

Squashing an empty function yields an empty sequence: $s3 = \langle \rangle$.

Example 5 : Continuous Function (No Gaps)

$$\left| \begin{array}{l} f4 : \mathbb{N}_1 \rightarrow \mathbb{N} \\ s4 : \text{seq } \mathbb{N} \end{array} \right| \quad f4 = \{1 \mapsto 100, 2 \mapsto 200, 3 \mapsto 300\} \wedge s4 = \text{squashFunc}(f4)$$

When the function has no gaps ($\text{dom } f4 = 1..3$), `squashFunc` simply converts it to a sequence: $s4 = \langle 100, 200, 300 \rangle$.

Example 6 : Relationship Between Function land Sequence

For any partial function f with a continuous domain starting at 1, `squashFunc` is the identity.

$$\left| \begin{array}{l} \text{continuous_}N : \mathbb{N}_1 \rightarrow \mathbb{N} \\ s_continuous_N : \text{seq } \mathbb{N} \end{array} \right| \quad \text{dom } continuous_N = 1.. \#(\text{ran } continuous_N) \wedge s_continuous_N = \text{squashFunc}(continuous_N)$$

In this case, `continuous_N` is already a sequence, and `squashFunc continuous_N = continuous_N` as a sequence.

Example 7 : Squash Preserves Injectivity

If the input function is injective, the output sequence has no duplicates.

$$\left| \begin{array}{l} \text{inj_func} : \mathbb{N}_1 \rightarrow \mathbb{N} \\ \text{inj_seq} : \text{seq } \mathbb{N} \end{array} \right| \quad (\forall i, j : \text{dom } \text{inj_func} \bullet \text{inj_func}(i) = \text{inj_func}(j) \Rightarrow i = j) \wedge \text{inj_seq} = \text{squashFunc}(\text{inj_func})$$

Since `inj_func` is injective (one-to-one), `inj_seq` contains no duplicate elements.

Example 8 : Practical Use Case - Video Database

$[VideoID, Title]$

$$\left| \begin{array}{l} \text{sparse_catalog} : \mathbb{N}_1 \rightarrow (VideoID \times Title) \\ \text{compact_catalog} : \text{seq } (VideoID \times Title) \end{array} \right| \quad \text{compact_catalog} = \text{squashFunc}(\text{sparse_catalog})$$

A sparse catalog with gaps (deleted entries) can be compacted into a continuous sequence while preserving the order of remaining entries.